No Refund for the Token Tax: Speculative Decoding Across 21 Languages

Hanyu Yang (ORCID 0009-0005-0419-4070)

October 2026

Abstract

Most languages pay a token tax: when Qwen3.5 translates the same sentences into Khmer, its output runs 4.7× as many tokens as its English output. Speculative decoding might pay part of it back. Because tokenizers split words into multiple segments, later segments should be easier to predict, allowing a lightweight drafter to achieve higher acceptance in high-fertility languages. We tested this on 21 languages using the built-in drafters of Qwen3.5-9B and Gemma 4 under fixed decision rules; there was no refund — English got the most tokens accepted per step. While continuation tokens offer a slight boost, the drafters guess worse outside English for almost every kind of token. In real serving with vLLM, speculation makes Gemma's English replies 2.4× faster than baseline, whereas Finnish replies are only 1.2× faster, expanding Finnish latency from 1.6× to 3.0× the English latency. The data and code are publicly available.

1 Introduction

The token tax. Language models process tokens rather than words. Tokenizers stem largely from English text, so most other languages are cut into more, smaller pieces. In our runs, Qwen3.5's Khmer translations of 200 FLORES-200 sentences [18] take 4.7 times as many tokens as its English ones, and its Amharic ones 4.5 times. Petrov et al. [1] and Ahia et al. [2] demonstrated consequences such as higher per-token fees, slower responses, and reduced usable context.

The hope. At Google, inference used about 60 percent of machine-learning energy in sampled weeks of 2019–2021 [3]. Workloads generate long outputs (reasoning traces, agents, and RL rollouts), among which rollouts dominate post-training time [4, 5]. Speculative decoding is the standard solution: a smaller model drafts several tokens, and the larger target model evaluates them in a single forward pass [6, 7]. Serving frameworks such as vLLM and SGLang support the feature, and DeepSeek-V3, Qwen3.5 and Gemma 4 provide built-in drafters [8, 9, 10].

The fertility refund. High-fertility languages—those requiring many tokens per word—might recover part of their tax. Because tokenizers split words into multiple segments, later segments are easier to predict once the first piece is known. The drafter should handle those continuations reliably, enabling the target model to accept more tokens per verification step. We call this hypothesis the *fertility refund*.

What we find. There is no refund; there is a surcharge. Evaluating two built-in drafters across 21 languages through exact replay and pre-written decision rules, acceptance length (AL) did not increase with fertility: it fell for Gemma, and stayed flat for Qwen, whose result is less certain. All 20 non-English languages ranked lower than English (Figure 1).

- The mechanism: Non-English text has many more continuation tokens, and these are easier to draft. That mechanism is real, but its impact is modest. Drafters lose far more because, outside English, they guess worse on word starts, continuations and most other tokens (Section 4.2).
- Real serving: The gap shows up in wall-clock time. In vLLM serving, English benefits the most from speculation: Gemma achieves a speedup of 2.38× for English and 1.23× for Finnish. Without speculation, generating a Finnish response takes 1.6× as long as English; with speculation, Finnish takes 3.0× as long (Table 1, Figure 3).

Related work. Yi et al. [12] and Do et al. [13] trained drafters for specific languages, and UniSpec [14] speeds up many languages without a trained drafter. Hong et al. [20] add a language-specific output head for over-split languages. Sandler et al. [15] and Paudel et al. [16] measured per-language acceptance using separate draft models; Paudel et al. used a Qwen3.5-9B verifier to evaluate 11 low-resource languages. Syvokon [19] shows that a dictionary-based drafter gains more where fertility is higher. None of these asks whether the built-in drafters that ship with today's models win back the token tax relative to English. We do, using exact replay and timed serving.

Why it matters. Speculative decoding benchmarks focus heavily on English outputs [11], so reported speedups overstate the benefit in most of the languages we tested. Two steps would help: train drafters on multilingual target outputs, and report speedups for each language.

Contributions.

1. Exact measurements of speculative acceptance across 21 languages for two built-in drafters, guided by decision rules set in place before the runs.
2. An exploratory split showing that the token-mix gain is small, and a drafting deficit within each token type outweighs it.
3. Real-world checks: vLLM latency and sampling tests, plus two EAGLE-3 heads. The built-in drafters' surcharge persists in serving; a head trained on Hindi and Gujarati reverses it for Hindi.
4. Open artifacts: the raw records of the reported runs, analysis code, and a self-checking reproduction script, at doi:10.5281/zenodo.23355847.

2 Background

Speculative decoding. A drafter proposes K candidate tokens. The target evaluates all K tokens in a single forward pass, retains the longest prefix it agrees with, and then appends one token of its own. In exact arithmetic, greedy output is unchanged [6, 7]; in vLLM it sometimes differs (Section 6). Acceptance length (AL) is the average number of tokens one verification step produces. Because a step that confirms a draft tokens produces $a + 1$ tokens, the acceptance length satisfies $\mathrm{AL} \geq 1$.

Built-in drafters. DeepSeek-V3 and Qwen3.5 train their multi-token prediction (MTP) head jointly with the model [8, 9]. Gemma 4 has a separate MTP drafter, released as its "assistant" checkpoint [10]. EAGLE-3 heads are trained after extracting the target features [17].

Fertility. We measure fertility as the token ratio T_L/T_{en}: how many tokens the target's own translations into language L take, against its own English translations of the same sentences. This mixes how the tokenizer splits text with how long the model's translations are (Appendix A).

3 Setup

- Models: We evaluate Qwen3.5-9B, which features a single-layer MTP head applied recursively, alongside Gemma 4 E4B-it with its MTP drafter. Both run in bf16 on a single A100.
- Data: The first 200 sentences of the FLORES-200 devtest [18], in 21 languages chosen to span fertility. The task is translation from English into each language. The English outputs are translated from French, so English also differs in source and direction.
- Measurement: The target first generates its own greedy translation y. We then run the drafter at every position of y to depth 5 and count how many drafts the target accepts for $K = 1, \ldots, 5$. This replay gives AL without timing noise.
- Check against vLLM: The English AL of vLLM at $K = 3$ is within 4% of the replayed value for both models. With speculation at $K = 3$, vLLM's greedy outputs match the Hugging Face outputs on all 20 English test prompts per model.
- Decision rules, set in place before the runs:
  - F1: the partial Spearman rho between AL at $K = 3$ and the ratio T_L/T_{en}, controlling for how well the target knows each language, does not exceed 0.10.
  - F2′: the median gap between step-1 acceptance of continuation tokens and of word-start tokens remains below 0.20, over languages that put spaces between words (Appendix A).
  - Every wiring component and validation gate needs to succeed.
  - Point estimates decide, and the refund hypothesis is rejected if either rule fires across both model families.
  - We documented the rules before the runs but they lack external timestamps; one change came after data: run 1 failed Qwen's wiring check, and we swapped in a weaker comparison, which made that check easier to pass.
- Serving: We deployed vLLM 0.31.0 on a single A100 40GB:
  - latency at batch size 1 across six languages, 30 sentences each;
  - sampling at $T \in \{0, 0.7, 1.0\}$ across all 21 languages, 100 sentences each;
  - two EAGLE-3 heads on Qwen3-8B: a general head trained on ShareGPT and UltraChat chat data, and a "multilingual" head trained only on Hindi, Gujarati, and English [21].

Appendix A gives the exact definitions.

4 Results

4.1 No refund

All gates pass in both families, and both rules fire in both.

- Qwen:
  - F1 $\rho = -0.033$: flat.
  - F2′ $= 0.172$: continuation tokens help, but by less than the 0.20 bar.
- Gemma:
  - F1 $\rho = -0.656$: acceptance falls with fertility.
  - F2′ $= 0.049$.
- AL at K = 3:
  - Qwen: English 3.16, other languages 1.89–2.81.
  - Gemma: English 2.93, other languages 1.26–2.19.
- Refund fraction: It is negative in all 14 languages where it is defined: Qwen -0.86 to -0.10,

Gemma −1.65 to −0.66. Zero would mean no refund, so these languages pay extra.

How sure are we? Gemma's result holds even when the language panel is resampled: F1 [−0.80, −0.40], F2′ [0.014, 0.137]. Qwen's does not: its language-resampled intervals cross both thresholds (F1 [−0.53, 0.52], F2′ [0.152, 0.224]). The rules use point estimates, so Qwen's verdict stands, but it is the weaker half.

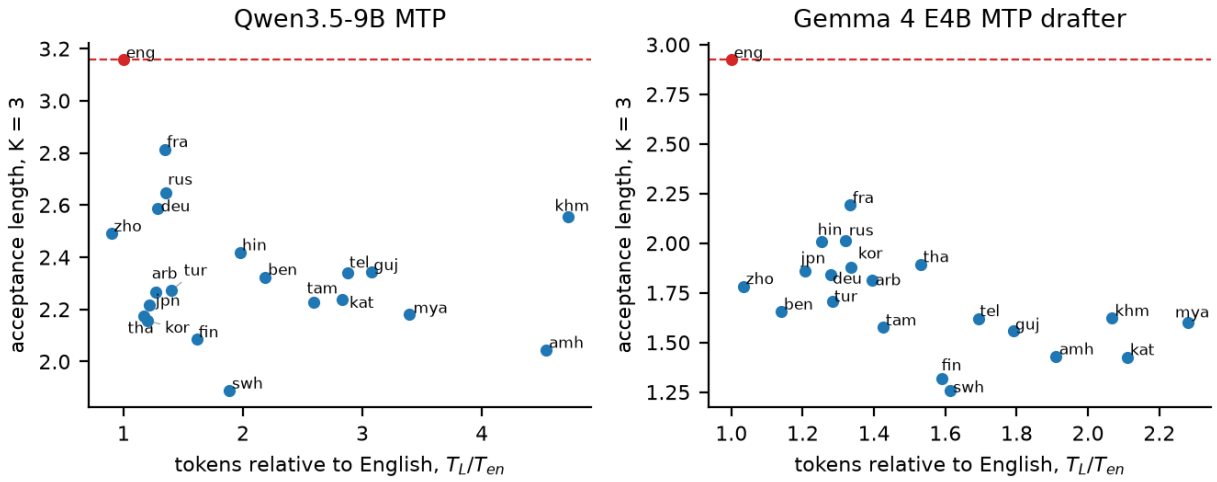


Figure 1: AL at $K = 3$ against the token ratio, one point per language. The dashed line is English.

4.2 Why: the mix helps, but the drafter reduces acceptance

We partition each language's step-1 acceptance gap relative to English into two components using a two-fold Oaxaca decomposition [22]:

- Mix: The portion attributable to different token-type distributions, measured by English acceptance rates.
- Within-type: The portion attributable to lower drafter acceptance within the same token type.

The results:

- The mix component is small and positive: on average over 20 languages, it adds 2.6 percentage points for Qwen and 5.5 for Gemma.
  - Non-English text contains substantially more continuation tokens, ranging 0.19–0.82 versus English 0.08 for Qwen, and 0.16–0.87 versus 0.06 for Gemma.
  - So the refund mechanism is real, but small. It tracks the contrast with English rather than fertility.
  - Qwen's Amharic is the exception: a third of its tokens are byte fragments, which English never produces, and its mix part is negative.
- The within-type drafting deficit is large and negative: −19.5 percentage points (Qwen) and −42.6 (Gemma), remaining below zero across all 20 languages.
- The deficit persists with depth: When draft 1 is accepted, Qwen's draft 2 is accepted 46–74% of the time outside English, against 85% in English (for Gemma, 43–66% against 83%).

This split covers the first draft only, so it explains the acceptance gap, not the full AL.

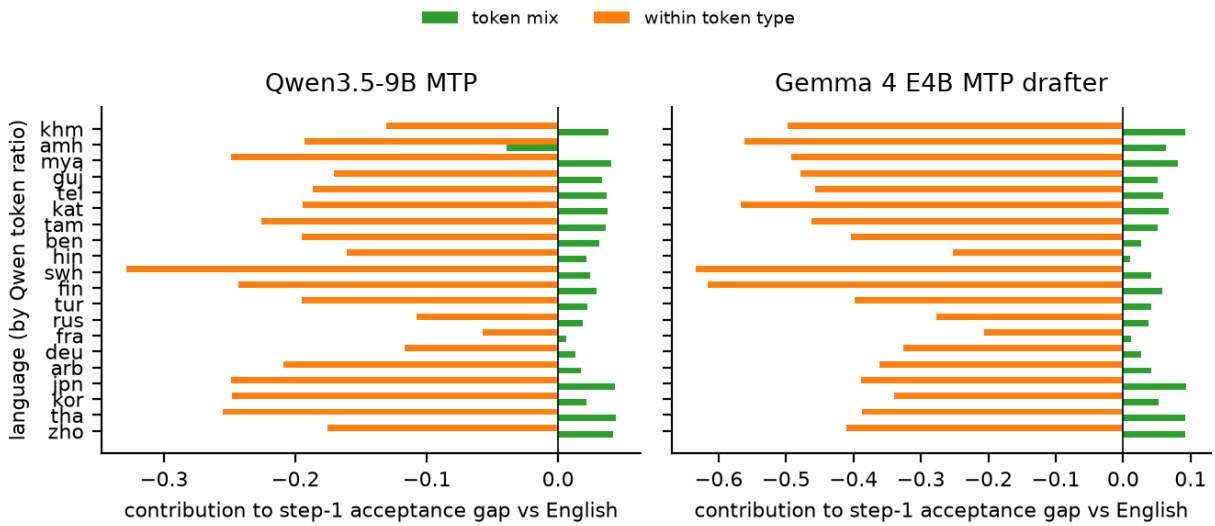


Figure 2: Each language's step-1 acceptance gap to English, split into the token-mix part and the within-type part. Both panels list languages in order of Qwen's token ratio.

4.3 In milliseconds

Table 1: Speedup at $K = 3$, batch size 1, vLLM 0.31.0, one A100 40GB. Each cell divides the time taken without speculation by the time taken with it, summed over the sentences.

speedup at K = 3	eng	zho	rus	fin	hin	amh
Qwen3.5-9B MTP	1.94	1.60	1.73	1.52	1.65	1.41
Gemma 4 E4B MTP drafter	2.38	1.54	1.86	1.23	1.78	1.30
Qwen3-8B + EAGLE-3 (general)	1.97	1.03	0.99	1.01	1.08	0.92†
Qwen3-8B + EAGLE-3 (multilingual)	1.31	0.92	0.95	0.93	1.64	1.25†

† Based on 3 Amharic sentences; the other 27 were invalid in one of the runs.

- The built-in drafters: For both, every non-English language gets a smaller speedup than English. The intervals do not overlap within this run (the serving report in the deposit lists them).
- The gap widens: The latency premium over English grows under speculation. Gemma's Finnish goes from 1.55× English latency to 3.02×. Across the six languages, speedup falls with fertility ($\rho = -0.54$ for Qwen, -0.71 for Gemma), but with six languages neither is significant.
- The general EAGLE-3 head gives almost nothing outside English (0.92–1.08).
- The multilingual head gives Hindi more speedup than English (1.64 against 1.31).
- These are for $K = 3$; the deposit also has $K = 1$ and $K = 5$.

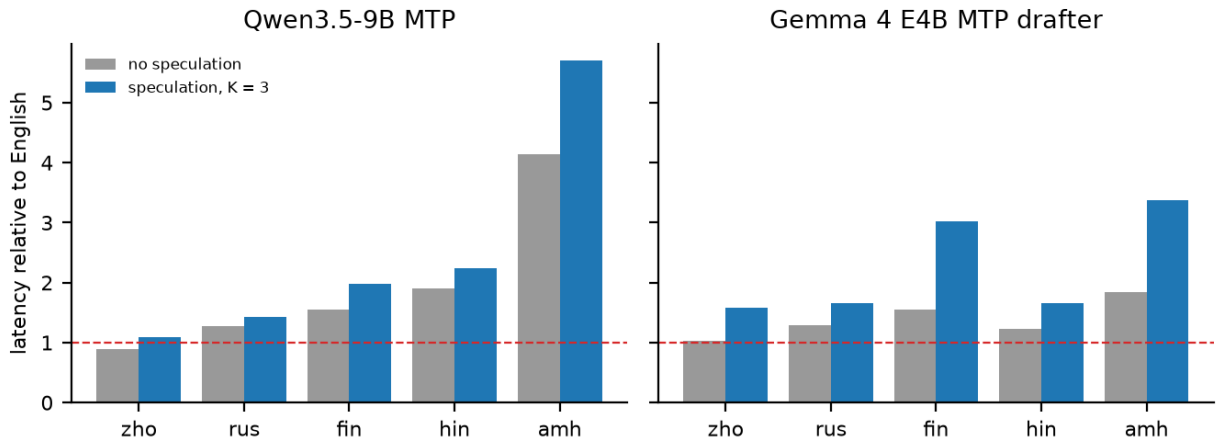


Figure 3: Latency relative to English, without speculation (grey) and with it at $K = 3$ (blue). The dashed line is parity with English.

4.4 Sampling and trained drafters

Table 2: Spearman ρ(AL, T_L/T_{en}) over 21 languages, $K = 3$, 100 sentences per language, from vLLM's own counters.

	T = 0	T = 0.7	T = 1.0
Qwen3.5-9B MTP	−0.21	−0.38	−0.48
Gemma 4 E4B MTP drafter	−0.68	−0.67	−0.67
EAGLE-3 general	−0.61	−0.65	−0.72
EAGLE-3 multilingual	0.02	−0.02	−0.02

- Sampling makes Qwen's trend more negative: English keeps the highest AL at every temperature for both built-in drafters. These values are not adjusted like F1, so Qwen's −0.21 at T = 0 is not a second estimate of F1.
- The general EAGLE-3 head falls with fertility, like the built-in drafters.
- The multilingual EAGLE-3 head is the exception:
  - Over all 21 languages its ρ is about 0.
  - Qwen3-8B produces degenerate (capped or looping) output for many sampled items in a few languages: 83 of 100 Amharic items at T = 0, and about half of Burmese. vLLM's counters cannot exclude those items.
  - Dropping languages above an invalid-item threshold between 5% and 50% (a check we added afterwards), its ρ ranges from +0.08 to +0.42 at T = 0 and from −0.13 to +0.15 at T = 1.0. The sign depends on the threshold. The general head stays at −0.57 to −0.73 at every threshold.
  - At T = 0, its English AL is 1.70, against 2.42 for the general head. It beats the general head in only 4 of 21 languages, mostly Gujarati (+1.20) and Hindi (+0.65).
- Takeaway: At T = 0 the multilingual head shows no downward trend, from a weak English start; at T = 1 the sign depends on the threshold. Besides language, the heads were trained on different data, by different recipes, with different draft vocabularies. The general head's draft vocabulary covers only about half of Hindi and Gujarati tokens [21], so some of the gap may be

missing words rather than worse guesses. So this hints that multilingual training helps; it does not show it.

5 What this means

- Report speedups per language: A single English number overstates the gain in most of the languages we tested. Our six-language latency panel took about one GPU-hour.
- Drafters are a fairness lever: Most of the gap sits within token types, which points at the drafter as well as the tokenizer, though our split cannot separate the two. Yi et al. [12], Do et al. [13] and Hong et al. [20] show that language-specific drafting helps.
- Inference-heavy workloads may amplify the gap: Speculative decoding is already used to speed up RL rollouts [4, 5]. If the per-language gap carries over to long reasoning traces, a speed gap becomes a cost gap. We measured translation only.

6 Limitations

- Scope:
  - One task (translation), FLORES sentences, two built-in drafters plus two EAGLE-3 heads on a third target.
  - One engine version and one GPU type. Latency is at batch 1 in six languages.
- What the token ratio measures: It comes from the models' own translations, and English is translated from French, so it mixes tokenizer splitting with translation behaviour.
- Statistics:
  - Sampling AL is a per-language aggregate that includes invalid items: 35 and 3 of 6,300 for Qwen and Gemma, 419 and 420 for the Qwen3-8B EAGLE arms. A long looping output adds many steps to the count, and removing whole languages gives a different language set rather than a cleaner one.
  - The decomposition and the threshold sweep were chosen after the verdict.
- The multilingual EAGLE-3 head: Its training data, per its model card, are Qwen3-8B responses to FLORES-200 prompts in Hindi, Gujarati and English. The card does not name the split. Its Hindi and Gujarati gains may partly reflect overlap with our test set.
- The decision rules: They were stated before the runs but are not externally timestamped, and one gate amendment followed the first run.
- The vLLM serving numbers: They come from one run; an earlier complete run was lost before its records were saved. In vLLM, speculative greedy output is often not identical to plain greedy output (down to half of Qwen's Amharic outputs at $K = 1$), and we did not compare output quality. So speedups are latency ratios over the same inputs.
- Records: The main run did not record model revisions or code fingerprints; the deposit lists the revisions. The shipped runner and verdict code are later versions with the same measurement.

Data and code

Data and code are at doi:10.5281/zenodo.23355847 (all versions: doi:10.5281/zenodo.23331652; CC BY-SA 4.0, code MIT). `reproduce.sh` recomputes the verdict, the decomposition and the serving

analyses from the raw records, and fails loudly on any mismatch. The deposit README lists what it does not cover.

Disclosure

This paper is AI-assisted. Claude Opus 5.5 (Anthropic) wrote the measurement, serving and analysis code and ran the experiments under my direction, and I checked the results. The data deposit was reviewed by GPT-6 Luna (OpenAI, through Codex), Gemini 3.1 Pro (Google) and a second Claude Opus 5.5 instance. This version of the paper was reviewed by Gemini 4 Argon (Google) and ChatGPT Pro (OpenAI). Claude Opus 5.5 checked the citations, made the figures, and drafted the prose from my notes. I rewrote and edited that draft. I take responsibility for the paper.

Appendix A: Methods details

- Token ratio, exactly: totals over the valid sentences, not an average of per-sentence ratios.
- Valid items: An output is dropped if it hits the 400-token limit or keeps repeating itself (more than 30% repeated 4-token chunks). A language counts only if at least 100 of its 200 sentences are valid in it and in English.
- AL: All output tokens except the first, divided by the number of verification steps.
- Refund fraction: $R_L = 1 - \ln(P_L/P_{\text{en}})/\ln(T_L/T_{\text{en}})$, where P is one plus the number of verification steps. It is 1 for a full refund, 0 for none, and negative for a surcharge. We compute it only where $T_L/T_{\text{en}} \geq 1.3$: 14 languages per model.
- F1's control: How likely the target finds the FLORES reference translation, per character. It measures how well the model knows the language, not the quality of its own output.
- F2′ languages: 16 per model, English included, each with at least 200 tokens of both kinds. Burmese, Chinese, Japanese, Khmer and Thai have no spaces and are left out.
- Token types: A word is a run of letters, marks or digits. A word-start token begins a word, a continuation token only extends one, and a byte fragment completes no character. Everything else is "other" or "special". In languages without spaces, a "word" can span several real words.
- Byte fragments in the split: English has none, so there is no English rate for them. They use the language's own rate and count fully as mix.
- Replay accuracy: Scoring the output in one pass picks the same token the target generated at 99.5% (Qwen) and 99.6% (Gemma) of positions, and the batched drafter matches an unbatched reference at 159 of 160 checked positions.
- Intervals: Item-resampled 95% intervals (1,000 resamples for the replay, 2,000 for serving) hold the language panel fixed. Language-resampled intervals ask how much the panel itself matters. All serving intervals are within one run.
- Latency premium: Mean latency in a language over mean latency in English.
- Language codes: amh Amharic, arb Arabic, ben Bengali, deu German, eng English, fin Finnish, fra French, guj Gujarati, hin Hindi, jpn Japanese, kat Georgian, khm Khmer, kor Korean, mya Burmese, rus Russian, swh Swahili, tam Tamil, tel Telugu, tha Thai, tur Turkish, zho Chinese.

References

1. A. Petrov, E. La Malfa, P. H. S. Torr, A. Bibi. Language model tokenizers introduce unfairness between languages. NeurIPS 2023. arXiv:2305.15425.
2. O. Ahia, S. Kumar, H. Gonen, J. Kasai, D. R. Mortensen, N. A. Smith, Y. Tsvetkov. Do all languages cost the same? Tokenization in the era of commercial language models. EMNLP 2023.
3. D. Patterson et al. The carbon footprint of machine learning training will plateau, then shrink. IEEE Computer 55(7), 2022. arXiv:2204.05149.
4. Cheng et al. Fast LLM post-training via decoupled and fastest-of-N speculation. arXiv:2511.16193, 2025.
5. Liu et al. SPEC-RL: Accelerating on-policy reinforcement learning with speculative rollouts. arXiv:2509.23232, 2025.
6. Y. Leviathan, M. Kalman, Y. Matias. Fast inference from transformers via speculative decoding. ICML 2023.
7. C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, J. Jumper. Accelerating large language model decoding with speculative sampling. arXiv:2302.01318, 2023.
8. DeepSeek-AI. DeepSeek-V3 technical report. arXiv:2412.19437, 2024.
9. Qwen Team. Qwen3.5-9B model card. `https://huggingface.co/Qwen/Qwen3.5-9B`, 2026.
10. Gemma Team. Gemma 4 technical report. arXiv:2607.02770, 2026.
11. H. Xia et al. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. Findings of ACL 2024.
12. Yi, Kim, Jeung, Chang, Yun. Towards fast multilingual LLM inference: Speculative decoding and specialized drafters. EMNLP 2024.
13. Do, Le, Nguyen. AdaSpec: Adaptive multilingual speculative decoding with self-synthesized language-aware training and vocabulary simplification. AAAI 2026.
14. Do, Le, Nguyen. UniSpec: Training-free speculative decoding for robust LLM acceleration across languages and hardware. ACL 2026.
15. Sandler, Üstün, Romanelli, Hooker, Fioretto. The disparate impacts of speculative decoding. arXiv:2510.02128, 2025.
16. Paudel, Ginn, De Nardi, Palmer. Speculative decoding and the curse of multilinguality. arXiv:2605.30580, 2026.
17. Y. Li, F. Wei, C. Zhang, H. Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. NeurIPS 2025.
18. NLLB Team et al. No language left behind: Scaling human-centered machine translation. arXiv:2207.04672, 2022.
19. O. Syvokon. Dictionary-based speculative decoding for non-Latin-script languages. UNLP 2026.
20. J. Hong, G. Lee, J. Cho. Accelerating multilingual language model for excessively tokenized languages. Findings of ACL 2024.
21. SwitchXDDD/multilingual-eagle3-qwen3-8b, `https://huggingface.co/SwitchXDDD/multilingual-eagle3-qwen3-8b`, with code at `https://github.com/harshilpatel22/multilingual-eagle3-indic`, 2026. The general head is `https://huggingface.co/RedHatAI/Qwen3-8B-speculator.eagle3`.
22. R. Oaxaca. Male-female wage differentials in urban labor markets. International Economic Review 14(3), 1973.